AI Talent Hub ИТМО

Практикум по исследовательской и разработческой деятельности в ML.

Занятие 2.

1. План занятия

1. Что такое baseline как продукт
2. Из чего состоит хороший baseline
3. Почему важен простой пайплайн
4. Как организовать ML-репозиторий
5. Конфиги, окружение, seed’ы

2. Почему baseline вообще нужен

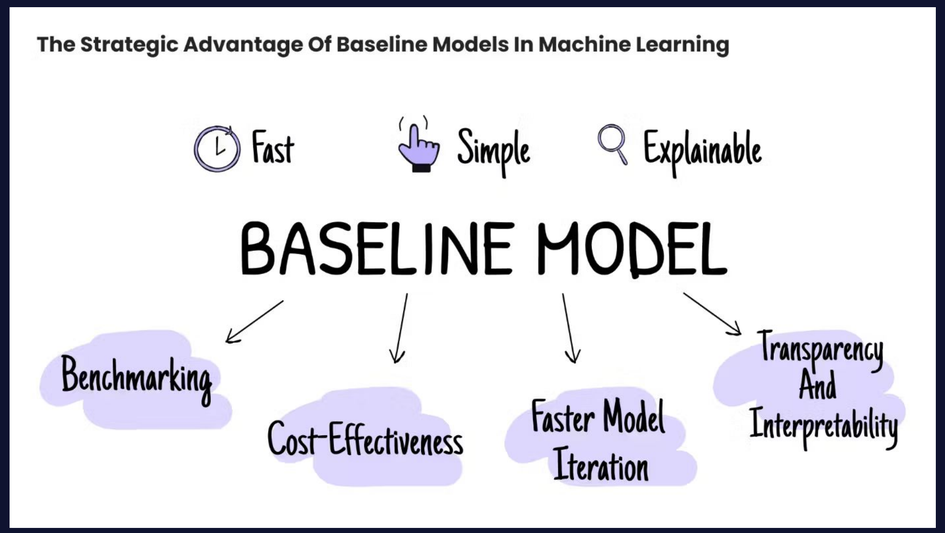
Baseline — это точка отсчёта.

Он нужен, чтобы:

- *проверить, что задача вообще решаема;*
- *проверить пайплайн end-to-end;*
- *сравнивать новые идеи честно;*
- *не обманывать себя случайными улучшениями.*

А также проверить:

- *работает ли сама постановка,*
- *есть ли сигнал в данных,*
- *не сломана ли метрика,*
- *не сделали ли вы leakage.*



3. Бейзлайн как продукт

Плохой baseline:

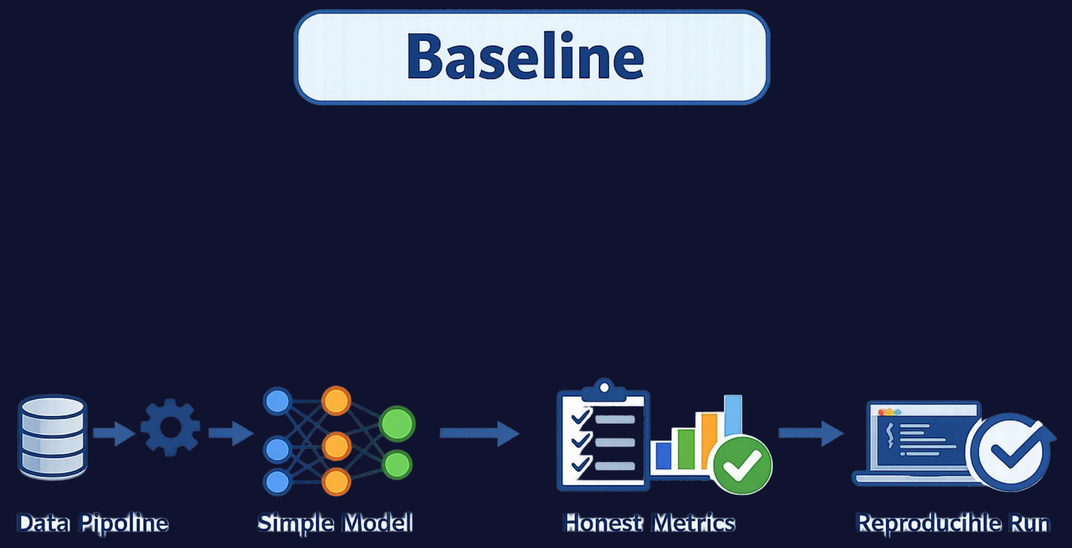
- “лишь бы что-то обучилось”

Хороший baseline:

- простая модель;
- простой пайплайн;
- честный, аккуратный evaluation protocol;
- воспроизводимый запуск;
- понятный output;
- оформленные артефакты.

4. “Формула” хорошего baseline

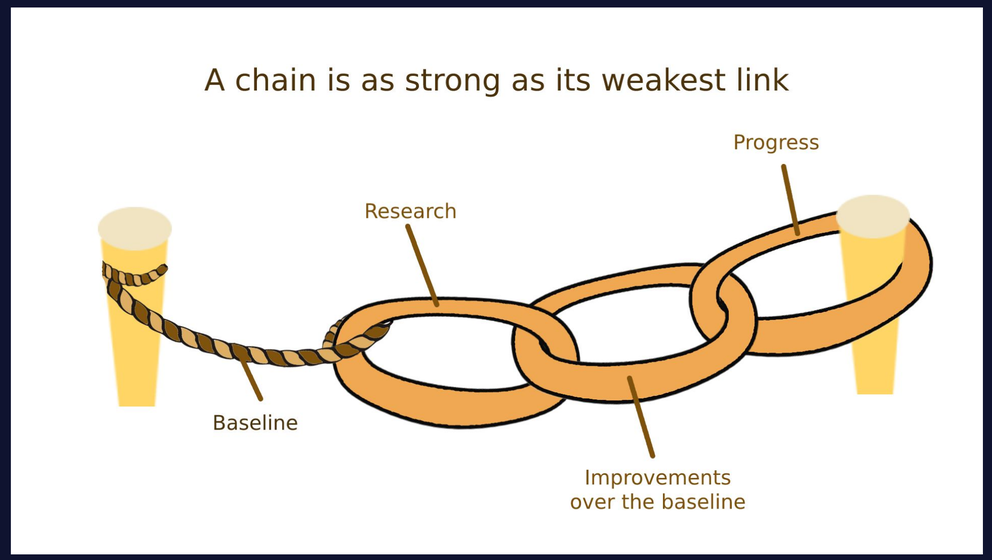
- нет честной метрики — нельзя доверять оценке;
- нет воспроизводимости — нельзя повторить результат;
- нет простого пайплайна — непонятно, где ошибка;
- нет простой модели — сложно интерпретировать, что происходит.



5. Примеры baseline для реальной задачи

Идея:

Baseline не обязан быть “слабым”, он должен быть простым, стабильным и понятным.



Примеры:

- табличка: Logistic Regression / CatBoost / LightGBM;
- текст: TF-IDF + Logistic Regression;
- CV: маленькая CNN / pretrained encoder + linear head;
- ranking: BM25 как baseline;
- LLM-задачи: prompt-based zero-shot/few-shot baseline.

6. Начинать с простой модели

Простая модель полезна, потому что:

- быстро обучается;
- легко дебажится;
- даёт нижнюю границу качества;
- помогает понять данные;
- дешевле по времени и ресурсам.

Baseline:

- *данные: текст + метка;*
- *split: train / val / test;*
- *модель: TF-IDF + Logistic Regression;*
- *метрика: macro F1;*
- *артефакты: метрики, confusion matrix, predictions.*

7. Код: простой baseline

8. Почему ML-проект без структуры быстро ломается

Без нормальной структуры обычно происходит:

- *всё в одном notebook;*
- *гиперпараметры захардкожены;*
- *train/eval/inference разбросаны по файлам;*
- *нельзя повторить запуск - другой человек не может запустить проект.*

```
bad-ml-project/
├── main.ipynb              # и загрузка данных, и обучение, и графики, и инференс
├── train.py                # часть train-логики
├── evaluate_model.py       # еще часть логики оценки
├── predict_test.py         # инференс отдельно
├── functions.py            # "все полезное"
├── final_functions.py      # "еще более полезное"
├── dataset.csv             # прямо в репозитории
├── dataset_v2.csv
├── model_final.pt
├── model_final_final.pt
├── results_new.csv
├── test2.csv
└── README.md               # "run main.ipynb"
```

8. Почему ML-проект без структуры быстро ломается

Не именно это дерево, скорее принципы:

- *код отдельно;*
- *конфиги отдельно;*
- *точки запуска отдельно;*
- *ноутбуки отдельно;*
- *артефакты отдельно.*

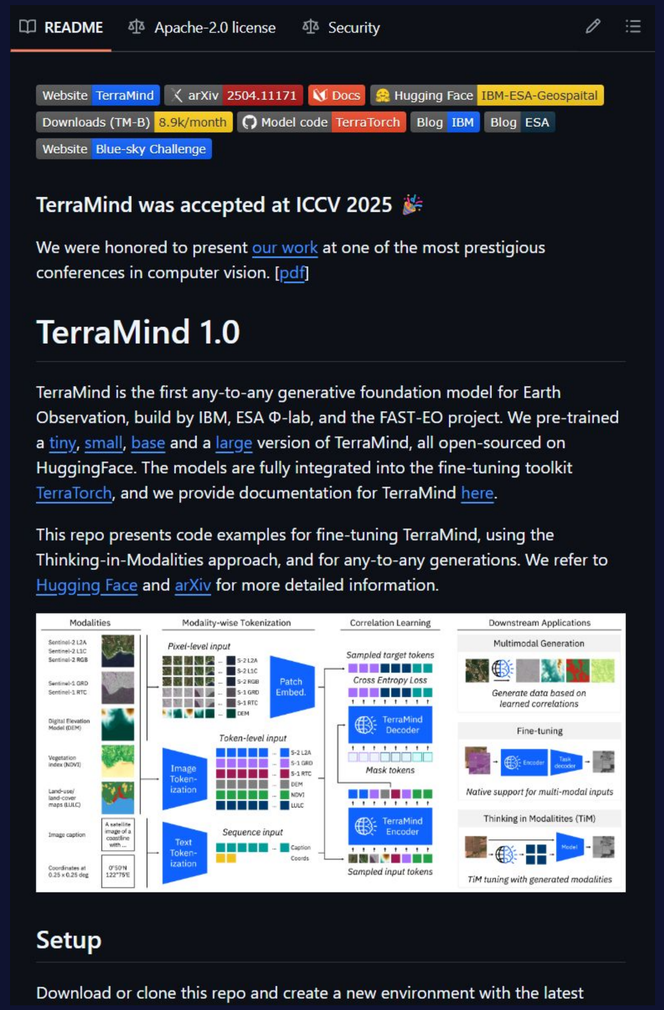
```
project/
├── README.md
├── pyproject.toml
├── requirements.txt
├── configs/
│   ├── train.yaml
│   ├── model/
│   ├── data/
│   └── experiment/
├── src/
│   ├── data/
│   ├── models/
│   ├── training/
│   ├── evaluation/
│   └── utils/
├── scripts/
│   ├── train.py
│   ├── evaluate.py
│   └── predict.py
├── notebooks/
├── tests/
├── artifacts/
└── logs/
```

9. Что обязательно должно быть в README

README должен отвечать на вопросы:

1. *Что решает проект?*
2. *Как поставить зависимости?*
3. *Как запустить train?*
4. *Как запустить eval / inference?*
5. *Где искать результаты?*

Если проект нельзя запустить по README, значит он не оформлен как инженерный артефакт.



README | Apache-2.0 license | Security

Website TerraMind | arXiv 2504.11171 | Docs | Hugging Face IBM-ESA-Geospaital
Downloads (TM-B) 8.9k/month | Model code TerraTorch | Blog IBM | Blog ESA
Website Blue-sky Challenge

TerraMind was accepted at ICCV 2025 🎉

We were honored to present our work at one of the most prestigious conferences in computer vision. [pdf]

TerraMind 1.0

TerraMind is the first any-to-any generative foundation model for Earth Observation, build by IBM, ESA Φ-lab, and the FAST-EO project. We pre-trained a tiny, small, base and a large version of TerraMind, all open-sourced on HuggingFace. The models are fully integrated into the fine-tuning toolkit TerraTorch, and we provide documentation for TerraMind here.

This repo presents code examples for fine-tuning TerraMind, using the Thinking-in-Modalities approach, and for any-to-any generations. We refer to Hugging Face and arXiv for more detailed information.

Setup

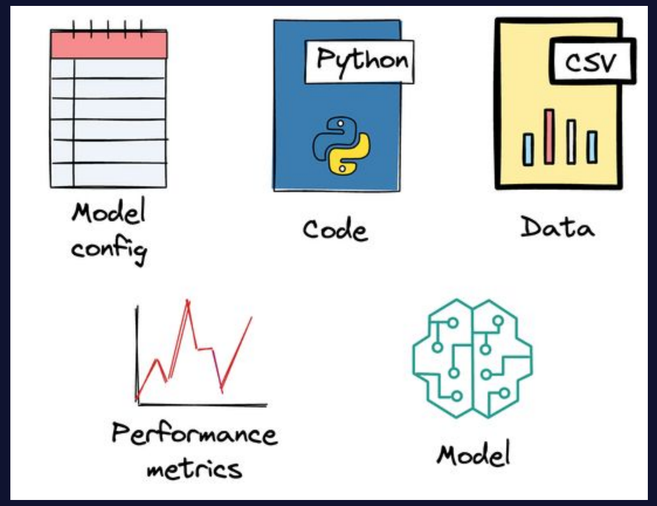
Download or clone this repo and create a new environment with the latest

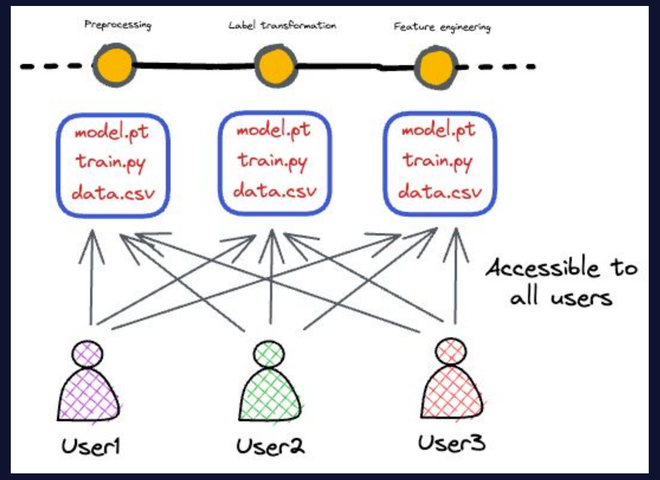
https://github.com/IBM/terramind

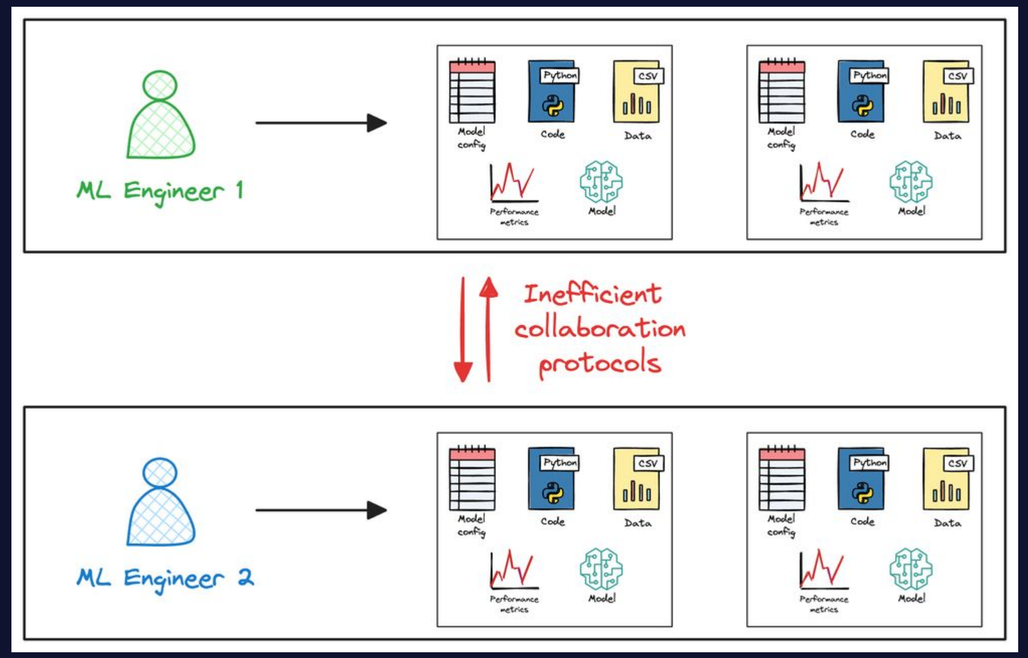
10. Конфиги

В конфиг выносят:

- пути к данным;
- параметры модели;
- batch size;
- learning rate;
- число эпох;
- seed;
- логирование;
- имя эксперимента.







11. Фиксирование окружения

Окружение — это тоже часть эксперимента.

Нужно фиксировать:

- *версию Python;*
- *версии библиотек;*
- *при необходимости CUDA / torch;*
- *способ установки зависимостей.*

```
numpy==1.26.4
pandas==2.2.1
scikit-learn==1.4.1
torch==2.2.1
hydra-core==1.3.2
wandb==0.16.4
```

```toml
TOML

[project]
name = "ml-baseline"
version = "0.1.0"
dependencies = [
    "numpy==1.26.4",
    "pandas==2.2.1",
    "scikit-learn==1.4.1",
    "torch==2.2.1",
    "hydra-core==1.3.2",
]
```

PDM

Poetry

uv
Python Package Manager

12. Seed’ы и воспроизводимость

Источники случайности:

- *инициализация весов;*
- *shuffle данных;*
- *stochastic optimization;*
- *data augmentation;*
- *недетерминированные CUDA-операции.*

```python
import random
import numpy as np
import torch

def set_seed(seed: int = 42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False
```

Для каждого run полезно сохранять:

- commit / версию кода;
- resolved config;
- seed;
- train/val/test метрики;
- веса модели;
- предсказания;
- версию датасета;
- графики и таблицы ошибок.

QA